

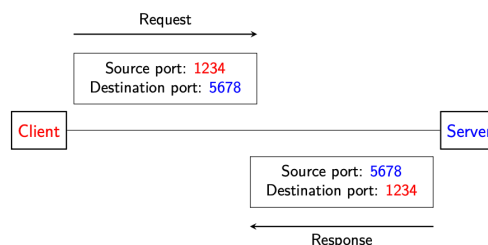
2-12

UDP user datagram protocol

fornisce un servizio di trasporto unreliable senza connessione su un unreliable servizio di connessione a network layer.

- ha una size massima di 65467 bytes
- non garantisce l'invio delle SDU
- ha error detection

il maggiore vantaggio è che permette a molte applicazioni che eseguono su un host di scambiare SDU con altre applicaizoni che eseguono su host remoti.



UDP header

contiene 4 campi:

16 bits di porta sorgente

16 bits di porta destinazione

16 bits length field

16 bits checksum

il numeri delle porte sono codificati il 16 bit quindi ci possono essere solo fino a 65535 processi legati a diverse porte UDP diverse allo stesso tempo in un dato server. per questo motivo le porte sono divise in maniera gerarchica:

- $1 < port < 1024$ solo per processi con privilegi di amministrazione di sistema
- $1024 \leq port < 49152$ protocolli che usano porte devono chiedere IETF un numero assegnato.
- $49152 \leq port \leq 65535$ tutte le applicazioni possono usarlo.

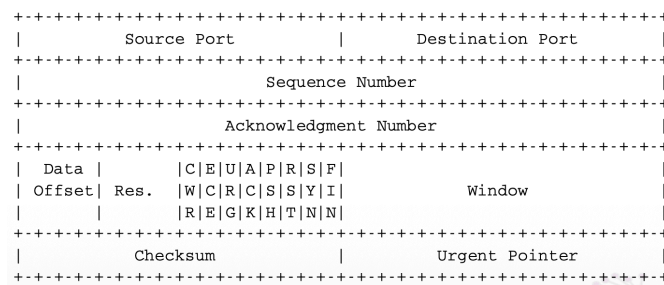


UDP usato per applicazioni che necessitano di minimizzare la latenza come DNS, QUIC, Voice over IP

TCP transmission control protocol

fornisce un sistema di bytestream orientato a connessione reliable, usato da un grande numero di applicazioni includendo web browsing, email, etc.

l'unità di trasporto del TCP è chiamato segmento, ogni byte corrisponde a un numero di sequenza.

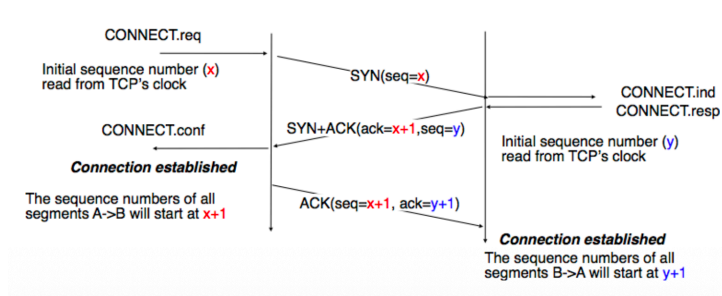


header

- 16 bits per le porte
- 32 bits per numero di sequenza e ack
- 4 bits Data offset indica la dimensione dell'header TCP in una parole lunghe max 32 bit
- reserved bits usati ora per ECN
- flags:
 - SYN per la connessione
 - FIN usato per la terminazione della connessione
 - RST chiude la connessione in caso di problemi
 - ACK indica che il campo di acknowledgment è valido
 - CWD, ECE per esplicita congestione di rete
- window: dimensione della finestra del ricevente
- checksum IP-style checksum su tutto il segmento TCP, e alcuni campi dell'header

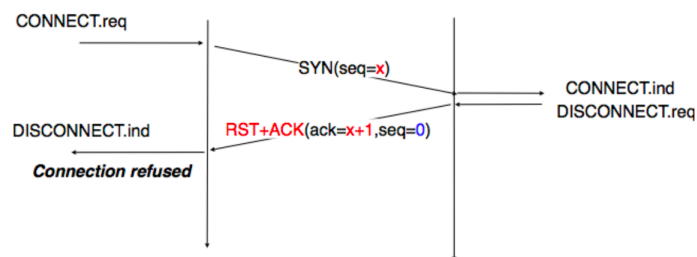
PSH and urgent pointer

PSH serve a forzare un flush del buffer del ricevente, questo non altera i numeri di frequenza dei segmenti, urge pointer fa la stessa cosa ma richiede l'invio del codice direttamente a livello applicazione senza considerare il numero di sequenza

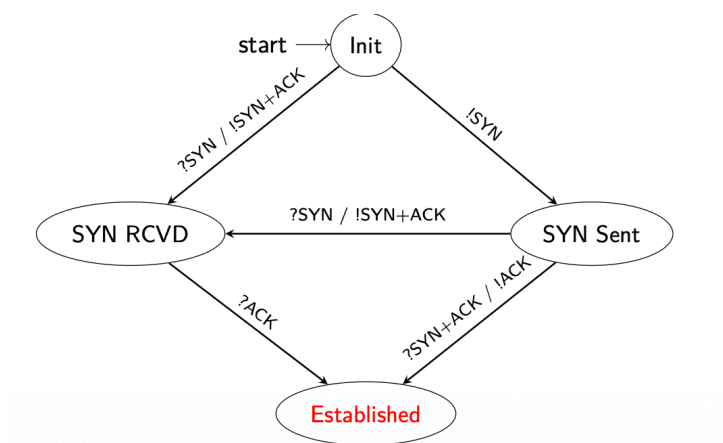


la differenza con una normale connessione TCP è l'interpretazione degli ACK: un ACK di $X + 1$ vuol dire aver ricevuto tutti i dati fino a X e ad aspettare $X + 1$

connection set-up



stabilimento della connessione



? significa ricevere, mentre !mandare

la novità è che due host possono essere entrambi client e server, se stabiliscono una connessione allo stesso momento usando esattamente le stesse porte, invece di rimanere in uno stallo perché mancherebbe la transizione di SYN RCVD e SYN sent mandano un SYN+ ACK che conta come ACK e permette alla connessione di stabilirsi comunque.

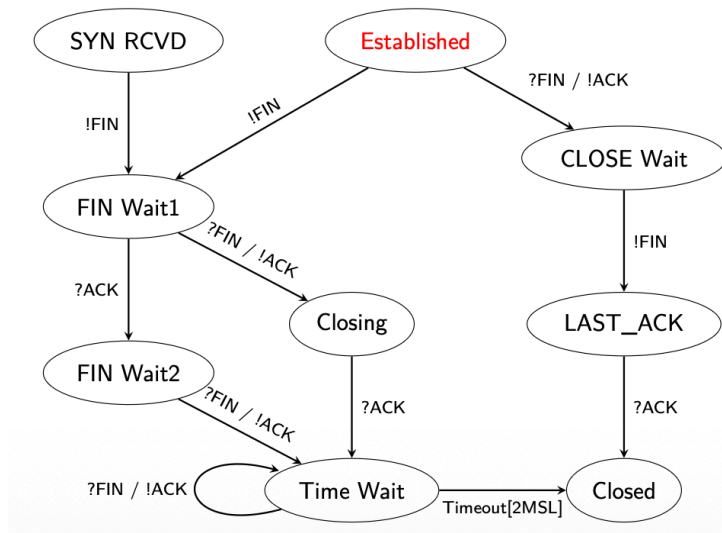
Il SYN e SYN+ACK può avere delle estensioni come MSS (max segment size), solitamente è impostato al MTU della rete (la dimensione massima che la rete del ricevente supporta prima della frammentazione, meno 40 Bytes, il minimo è 536, ma spesso è maggiore).

per implementare l'FSM entrambi gli endpoint devono mantenere uno stato, per comunicarsi tra di loro viene usato il TCB (transmission control block)

connection release

TCP supporta due tipi di release: Graceful e Abrupt;

- graceful avviene con l'invio del segmento FIN e del numero di sequenza X , il mandante può rimandare dati perduti in quanto il ricevente aspetta un FIN di risposta
- abrupt avviene con RST flag e quando viene mandato o ricevuto il TCB è cancellato



L'RFC richiede uno stato di time wait che deve essere mantenuto per $2 * MSL$ (4 minuti)

questo perchè da entrambe le parti l'ACK contenuto in ?FIN/!ACK potrebbe venire perso, in questo caso l'altro manderà un altro FIN che deve essere ACKato, questo però può portare a una larga occupazione di memoria.

esercizio

memoria dei TCBs: 32GB

la dimensione di un singolo blocco su linux è

```
cat /proc/sys/net/ipv4/tcp_rmem
4096 131072 6291456
```

con min/default/max

consideriamo default: $32/131072 = 262144$ connessioni

assumiamo che la connessione resti attiva per 1 minuto, quante connessioni posso accettare al secondo?

le connessioni devono rimanere in memoria per 1+4 minuti

questo vuol dire che il server può accettare un max di $262144/(60 * 5) \simeq 874$.

il server però sta usando l'80% della sua memoria per le connessioni in time_wait, su linux il timeout è settato a $3.5 * RTO$ (retransmission timeout)

TCB

contenuti statici:

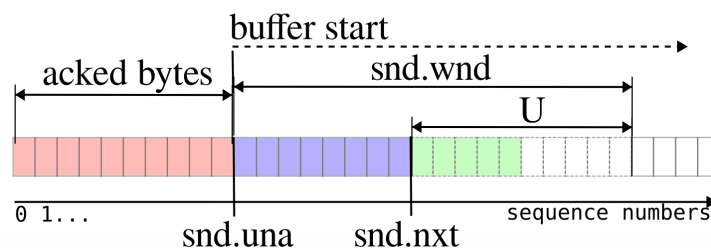
- IP locale e remoto, porte
- sending buffer di dati mandati ma non acknowledged

- receiving buffer per i dati ricevuti dall'host non ancora inviati all'user, sono memorizzati o perchè l'user è troppo lento a processarli o perchè i dati non sono stati ricevuti in sequenza.

contenuti dinamici:

- stato corrente FSM (fine state machine) → SYN-Sent, SYN-received, Established
- MSS (può cambiare) (max segment size)
- snd.nxt: numero del prossimo byte in spedizione
- snd.una → l'ultimo numero di sequenza mandato ma non ackato
- snd.wnd → dimensione della finestra buffer:

sending buffer



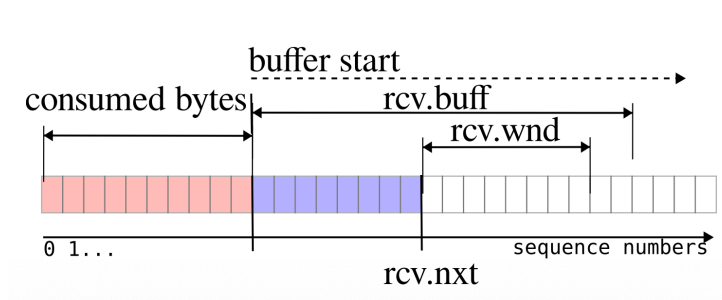
se l'applicazione deve MANDARE:

1. controlla se il sending buffer ha abbastanza spazio, se no manda errore
2. controlla inoltre che i dati nel sending buffer non abbia più dati della dimensione della finestra mandata dal ricevente
3. fino al MSS i dati sono messi nel payload del segmento
4. il sequence number di tale segmento è impostato a snd.nxt e il snd.nxt nel TCB è aumentato quanto la dimensione del payload del segmento TCP
5. l'ack number è settato a rcv.nxt e il campo della finestra segmento TCP è calcolato in base allo spazio libero nel receiving buffer
6. il segmento è mandato e i dati sono trattenuti nel sending buffer nel caso debbano essere rimandati

quando arriva un nuovo segmento:

1. setta snd.wnd al valore ricevuto nel segmento
2. compara il numero di ack con snd.una
3. se ci sono dati appena acknowledged li rimuove dal sending buffer e snd.una viene aggiornato

receiving buffer



se il segmento TCP contiene dati, il ricevente:

1. compara il numero di sequenza con `tcv.next`
2. se son uguale il segmento è stato ricevuto in sequenza, i dati possono essere inviati allo user e `rcv.next` aggiornato
3. il contenuto del receiving buffer è controllato per capire se altri dati sono già presenti, se lo sono viene riaggiornato `rcv.next`
4. il payload del segmento è messo nel receiving buffer
5. un ack è mandato al sender con la dimensione aggiornata della finestra

quando mandare i dati

subito ASAP

appena il sending buffer non è vuoto puoi mandare un segmento TCP, in alcuni casi questo approccio conviene, ma un segmento ha almeno 40B di header, (20 IPv4, 20 TCP) se mandi un byte per segmento 40/41 della capacità sono persi.

aspetta di avere MSS prima di mandare

in questo caso hai la massima efficienza, se la tua applicazione produce dati con un throughput alto questa è un'ottima scelta, non lo è se il tuo buffer impiega tempo per riempirsi, i dati al suo interno diventano vecchi prima di mandarli.

nagle algorithm

`snd.wnd` è settato al valore ricevuto dai frames di ack

```
# "data" is the block of bytes that has not been sent so far.
# It is made of new data just arrived, plus eventually unsent data in the buffer
if len(data) >= MSS and U >= MSS:
    send one MSS-sized segment
else:
    if there are unacknowledged data:
        place data into the sending buffer until acknowledgment has been received
    else:
        send one TCP segment containing at most snd.wnd data
```

| data sono quelli in verde del sending buffer

conseguenze

l'algoritmo di nagle fa in modo che è più probabile che vengano inviati segmenti grandi, la maggior parte dei pacchetti o sono 1460B o 0B, 1500 è l'MTU dell'ethernet link layer quindi la MSS è calcolata da quello, 0B sono gli ack, questo però lo rende non utile per il traffico real time.

dimensione della finestra

la dimensione della finestra è definita dal massimo numero esprimibile nell'header 2^{16} e non è possibile avere più di una finestra mandata per RRT (round trip time)

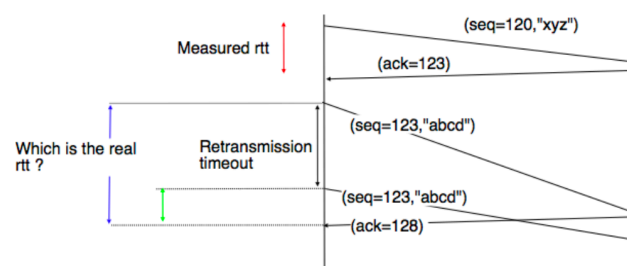
RTT	Maximum Throughput
1 msec	524 Mbps
10 msec	52.4 Mbps
100 msec	5.24 Mbps
500 msec	1.05 Mbps

per aumentare la capacità della finestra viene aggiunto un campo all'header: window scale option presente solo nel SYN e SYN-ACK e la dimensione della finestra diventa $rcv.wnd * 2^s$ con $0 \leq s \leq 14$

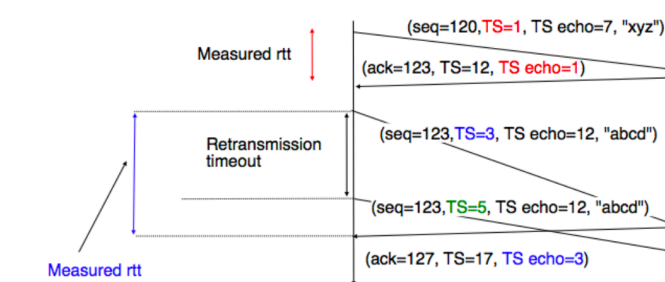
RTT	Maximum Throughput
1 msec	8590 Gbps
10 msec	859 Gbps
100 msec	86 Gbps
500 msec	17 Gbps

RTO retransmit time out

idealmente poco più grande del RTT(round trip time); il problema è misurare l'RTT che nelle connessioni lossy è difficile da misurare e cambia velocemente

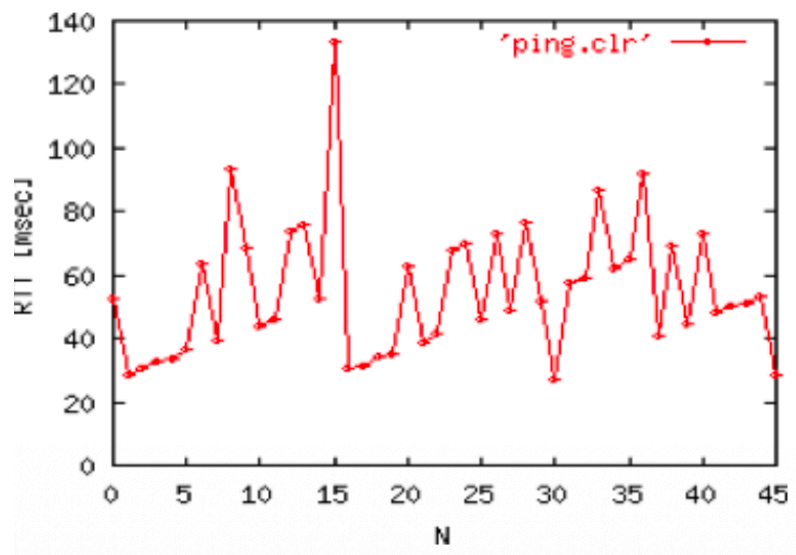


timestamps



viene aggiunto all'header e indicano il tempo corrente l'ultimo ricevuto dall'endpoint, in questo modo è facile calcolare il RTT. Questo risolve anche il problema delle reti veloci con numeri di sequenza a 32 bit dove era possibile avere 2 segmenti con lo stesso numero di sequenza a causa di delay, si vede subito che uno è vecchio.

ponendo che sia in grado di misurare la RTT come faccio ad arrotondare la misurazione



l'algoritmo che sene occupa tiene conto della sua variabilità sviluppato da Van Jacobs, tiene conto di 3 variabili:

rtt : l'ultima misurazione

srtt: l'RTT arrotondata, inizializzata con il primo valore di *rtt*

rttvar :deviazione stimata dal valore medio dell'RTT, inizializzato a *rtt*/2

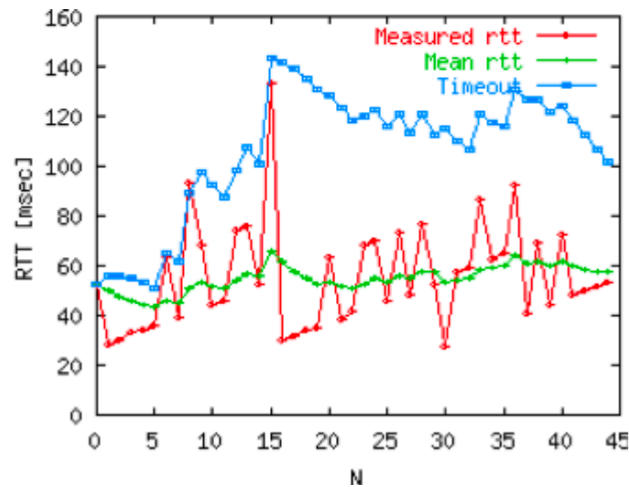
rto :retransmission timeout

con una serie di esempi s_i e un valore $0 < \alpha < 1$

■

si dice exponential moving avarage, (EMA) della sequenza s_i . Più $\alpha \simeq 1$ più conta l'ultima misurazione rispetto alla media calcolata fino a quel momento

algoritmo di Van Jacobson



i parametri descritti prima sono aggiornati come segue:

D

RTO EXP backoff: il RTO è raddoppiato per ogni tentativo fallito di trasmissione, dopo un successo viene resettato al valore iniziale. Se il ricevitore è in congestione non può gestire molti pacchetti, il mandante dovrebbe rallentare.

fast retransmit

il mandante potrebbe aver mandato fino a $\text{seq} = X$ ma ricevere un $\text{ACK} = Y < X$. Le ACK sono cumulative per cui ricevere un ACK per un numero di sequenza vecchio può voler dire che alcuni pacchetti sono stati persi, o che i due ACK sono stati invertiti e che $\text{ACK}(X)$ sta arrivando

Fast Retransmit prevede di ritrasmettere prima del termine del timer solo dopo aver ricevuto 3 copie dello stesso ACK vecchio.

delayed ACKs

Quando la comunicazione è per la maggior parte mono-direzionale il ricevente può mandare degli ACK dopo un certo numero di pacchetti ricevuti o un timeout, ma non integra bene l'algoritmo di Nagle in quanto go-back-n non è performante quando ci sono grandi perdite perchè la finestra non si muove.

selective Ack

con questo viene aggiunta un'estensione all'header dove il ricevente acknowledges blocchi di dati non in sequenza ricevuti, ad esempio $[1100 - 1200][1250 - 1500][1800 - 1900]$, conterrà anche l'ACK number nell'header TCP fisso ad esempio 850. in questo modo il mandante può interpretare che sono stati ricevuti tutti i bit precedenti a 850 e negli intervalli $[1100 - 1200][1250 - 1500][1800 - 1900]$